

EPISODIO 1

# Conectamos las piezas

Damos el primer paso: integramos HTML y PHP, entendemos los formularios y aprendemos a enviar datos con GET y POST.

▶ Comenzar episodio

+ Mi lista

⌚ Duración  
90 min

📊 Nivel  
Inicial

</> Tecnologías  
HTML · PHP

📝 Actividad  
TP práctico

## Qué veremos en este episodio



### 1. Integración HTML + PHP

Cómo PHP puede generar HTML y cómo HTML envía información a PHP.



### 2. Formularios

La etiqueta <form>, action, method y los controles de entrada.



### 3. GET

Enviar datos en la URL. Cuándo usarlo y cómo recibirlos con \$\_GET.



### 4. POST

Enviar datos en el cuerpo de la petición. Cuándo usarlo y cómo recibirlos con \$\_POST.



### 5. GET vs POST

Diferencias, ventajas y buenas prácticas para elegir el método correcto.



### 6. Validación

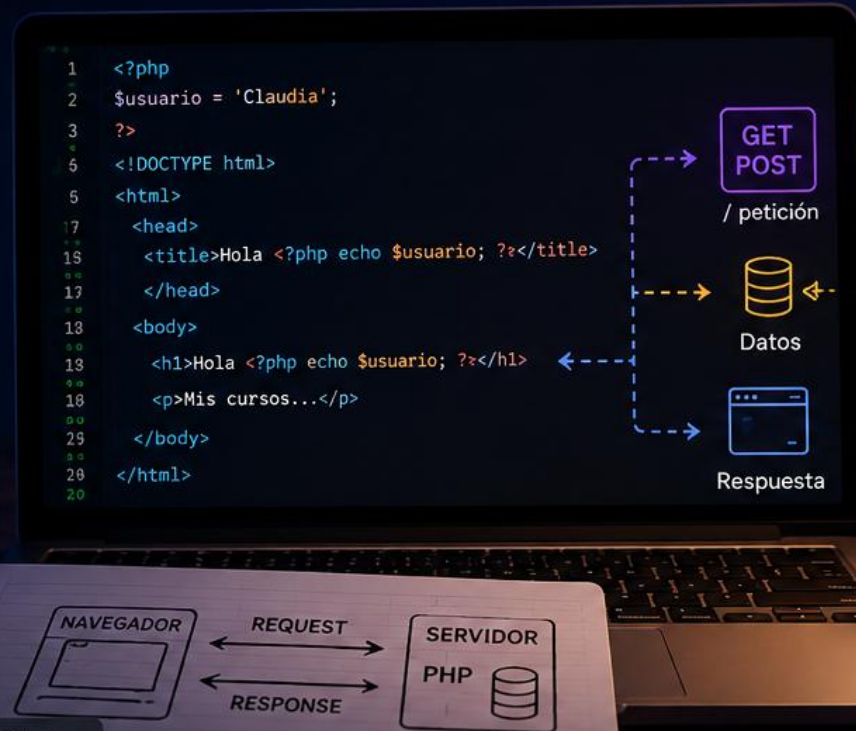
Validación en el cliente (HTML5/JS) y en el servidor (PHP).



### Actividad práctica (TP)

8 ejercicios para aplicar lo aprendido: formularios, GET, POST, radios, checkboxes, selects y más.

Ver consigna del TP >



## PROGRAMACIÓN WEB DINÁMICA

# De una página web a una aplicación web

Episodio 1 · Conectamos las piezas



# ¿Qué traemos en la mochila?

No empezamos de cero.

IP

PHP

PEyLW

HTML · CSS · JavaScript

IP / IPOO

PHP · Objetos · POO

Persistencia

SQL · Base de datos

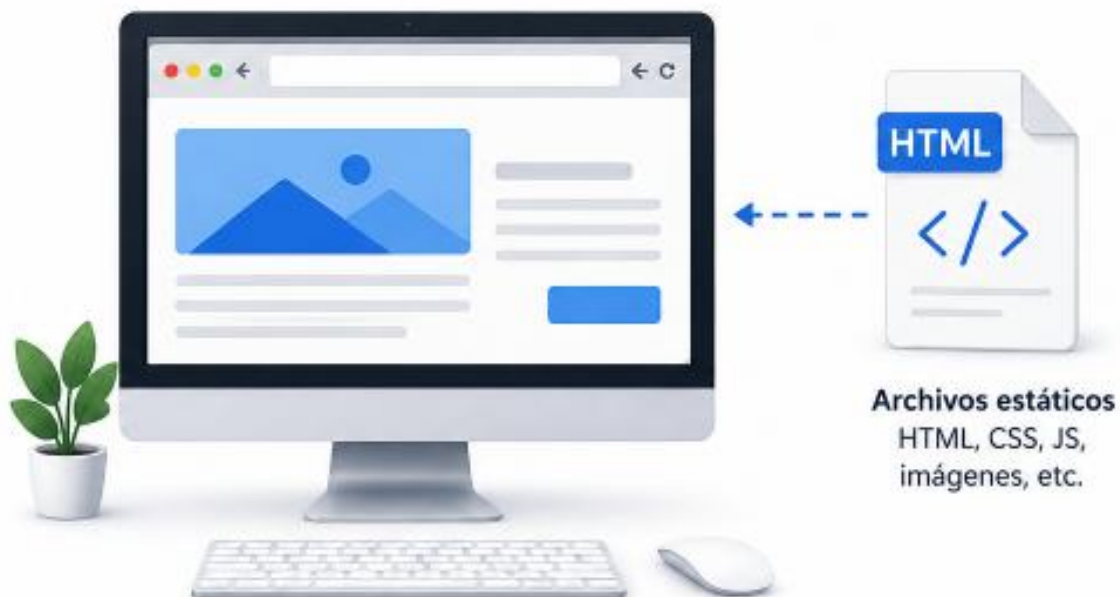
**Esta materia no agrega otra pieza: las hace trabajar juntas.**

# Lo que ya sabemos hacer

Una página puede vivir casi completamente en el navegador.

## PÁGINA WEB ESTÁTICA

Todo el código se ejecuta en el navegador del cliente.



### ¿Qué ocurre?

- 1 El navegador solicita un archivo (ej: index.html)
- 2 El navegador descarga los archivos estáticos (HTML, CSS, JS, imágenes)
- 3 El navegador interpreta y ejecuta el código (HTML, CSS, JS)
- 4 El resultado se muestra en la pantalla



### Características

- No requiere servidor ni base de datos.
- Siempre se comporta igual.
- No guarda estado.
- Ejemplos: sitios institucionales, portfolio, landing pages.



### Clave

Toda la lógica está del lado del cliente (en el navegador).

# Pero ahora quiero algo más...

Una aplicación responde distinto según el usuario, los datos y la acción.



## ¿De dónde salió mi foto?

¿Dónde están mis cursos?

¿Quién decidió qué mostrar?

¿Dónde se guardó esa información?

## Algo tiene que pasar del otro lado

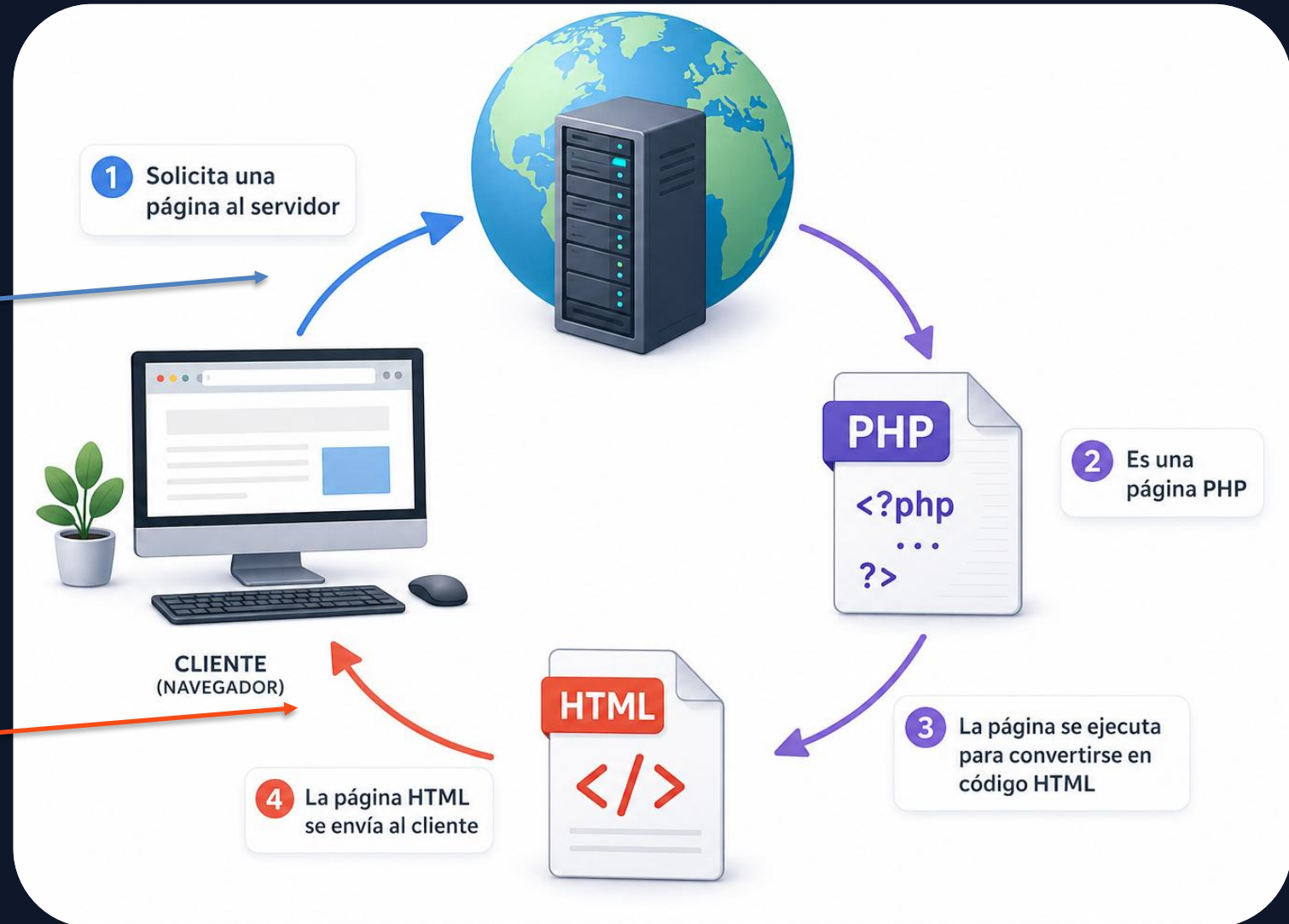


# Cliente ↔ Servidor

La conversación básica de la Web.

REQUEST

RESPONSE



# Escribimos una URL. ¿Qué acaba de pasar?

`https://pedco.uncoma.edu.ar/my/courses.php`

1 El navegador hace una petición

2 La petición llega a un servidor

3 La aplicación interpreta la petición y decide quién debe atenderla

4 La aplicación procesa la petición

5 La aplicación construye una respuesta

6 El navegador la muestra

# ¿Qué viaja realmente?

El navegador no recibe nuestro código PHP.

```
<?php
$usuario = buscarUsuario();
echo "<h1>Hola $usuario</h1>";
?>
```

PHP se ejecuta en el servidor



```
<h1>Hola Claudia</h1>
<p>Mis cursos...</p>
```

HTML / JSON vuelve como respuesta



# Entonces, ¿qué hace PHP?

---

**El PHP que conocían no desapareció.  
Cambió el contexto en el que lo usamos.**

## PHP EN EL SERVIDOR

Ejecuta lógica

Trabaja con objetos

Consulta / modifica datos

Genera una respuesta

**Y ahora produce  
respuestas para un  
cliente.**

# ¿Y nuestros objetos, dónde quedaron?

IPOO entra a la Web.



# Ahora sí tenemos todas las piezas

Este esquema nos va a acompañar durante varias clases.



# Página web ≠ aplicación web

No es una frontera perfecta, pero sirve para pensar.

---

## Predominantemente estática

Portfolio  
Landing page  
Información institucional  
Contenido similar para todos

## Aplicación web

Moodle / PEDCO  
Home banking  
Reservas · tienda · sistema académico  
Datos + usuarios + acciones

# Ahora hacemos conversar HTML y PHP

Hasta ahora conocíamos las dos piezas. Hoy las conectamos.

## CLIENTE · HTML

### Formulario

El usuario ingresa datos y realiza una acción.

REQUEST →

← RESPONSE

## SERVIDOR · PHP

### Procesamiento

PHP recibe esos datos, ejecuta lógica y construye una respuesta.

# Un formulario no es sólo una pantalla

Cuando hacemos clic en Enviar, el navegador construye una **petición HTTP**.

```
<form action="procesar.php" method="post">  
  <input type="text" name="nombre">  
  <input type="number" name="edad">  
  <button type="submit">Enviar</button>  
</form>
```

## DOS ATRIBUTOS CLAVE

**action**

¿A dónde enviamos la información?

**method**

¿Cómo viajan esos datos?

# GET · Los datos forman parte de la URL

```
<form action="buscar.php" method="get">  
  <input name="texto" value="laravel">  
  <button>Buscar</button>  
</form>
```



buscar.php?texto=laravel

**GET es ideal cuando estamos pidiendo / consultando información.**

La URL puede verse, copiarse, guardarse y modificarse.



# POST · Los datos viajan en el cuerpo de la petición

```
<form action="guardar.php" method="post">  
  <input name="nombre" value="Ana">  
  <button>Guardar</button>  
</form>
```



## PETICIÓN

|      |             |
|------|-------------|
| URL  | guardar.php |
| BODY | nombre=Ana  |

**POST** suele usarse cuando la petición envía datos para producir un cambio.

**Importante:** que no aparezcan en la URL  
**NO** vuelve secretos a los datos.

# GET vs POST · No es sólo «se ve / no se ve»

---

## GET

- ✓ Consultar / recuperar información
- ✓ Parámetros en la URL
- ✓ Puede compartirse o guardarse la URL
- ✓ Cambiar la URL cambia la consulta
- ✓ Ej.: búsquedas, filtros, páginas

## POST

- ✓ Enviar información para procesar
- ✓ Datos en el cuerpo de la petición
- ✓ No quedan expuestos en la URL
- ✓ No implica seguridad por sí mismo
- ✓ Ej.: altas, login, formularios

# ¿Cómo recibe PHP lo que mandó el navegador?

PHP expone los parámetros recibidos en arreglos asociativos.

## GET

```
// URL: procesa.php?edad=20

$edad = $_GET["edad"];
echo $edad;
```

## POST

```
// Formulario method="post"

$edad = $_POST["edad"];
```

La clave "edad" viene del atributo name="edad" del input.

# No todos los controles envían los datos igual

---

El TP usa varios tipos de controles. Conviene mirar qué recibe realmente PHP.

**text / number**

un valor

nombre=Ana

**radio**

un valor elegido

estudio=secundario

**select**

un valor elegido

operacion=suma

**checkbox**

puede enviar varios valores

deportes[]=futbol

# Cliente valida. El servidor también.

---

## EN EL NAVEGADOR

**HTML5**      required · min · max · type

**JavaScript**      reglas de interacción y validaciones adicionales

## EN EL SERVIDOR

**PHP**      vuelve a validar lo recibido

**¿Por qué?**      Porque el cliente se puede modificar o saltar.

**Regla que nos va a acompañar: nunca confiamos ciegamente en lo que llega.**

# Probemos mirando la petición

---

Antes de escribir más código: usemos el navegador como herramienta.

- 1 Enviar un formulario con GET
- 2 Mirar cómo cambia la URL
- 3 Modificar un parámetro directamente
- 4 Cambiar a POST y volver a enviar
- 5 Abrir DevTools → Network y comparar

# TP 1 · Hacemos que la página empiece a responder

El objetivo no es aprender sintaxis nueva: es conectar lo que ya saben.

## REPASAMOS

HTML + formularios CSS  
validación HTML5 / JS

PHP + arreglos  
condicionales

## INCORPORAMOS

action / method

GET y POST

\$\_GET / \$\_POST

name → clave

request → response

## OBSERVAMOS

qué viaja

dónde viaja

qué puede modificar el  
usuario

qué procesa PHP

**8 ejercicios · de un número simple a decisiones, controles múltiples y cálculo de tarifas.**

# Antes de irnos · ¿qué tiene que quedar claro?

---

- ? ¿Qué hace action en un formulario?
- ? ¿Qué diferencia conceptual hay entre GET y POST?
- ? ¿Cómo llegan los datos a PHP?
- ? ¿Qué relación hay entre name y \$\_GET / \$\_POST?
- ? ¿Por qué la validación del navegador no alcanza?

**Próximo episodio: cuando esto crece, necesitamos organizar responsabilidades.**



# Antes de irnos · ¿qué tiene que quedar claro?

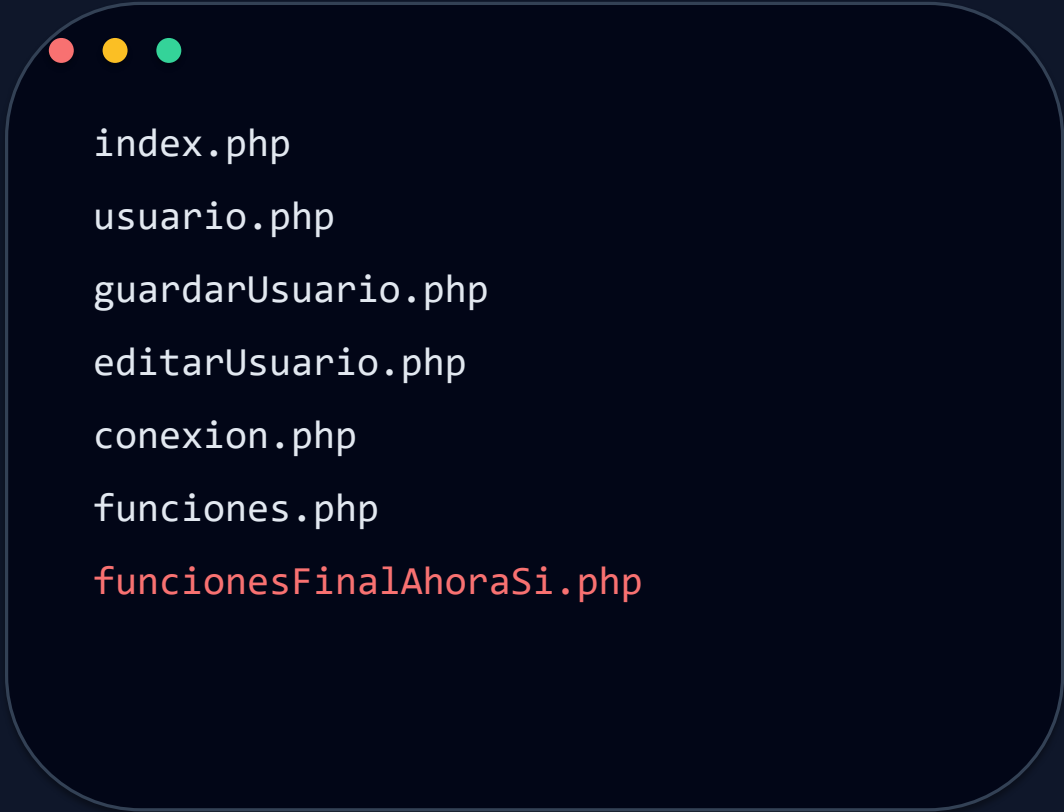
---

- ? ¿Qué hace action en un formulario?
- ? ¿Qué diferencia conceptual hay entre GET y POST?
- ? ¿Cómo llegan los datos a PHP?
- ? ¿Qué relación hay entre name y \$\_GET / \$\_POST?
- ? ¿Por qué la validación del navegador no alcanza?

**Próximo episodio: cuando esto crece, necesitamos organizar responsabilidades.**

# Pero tenemos un problema...

Cuando el proyecto crece, “hacer archivos” deja de ser una estrategia.



```
index.php
usuario.php
guardarUsuario.php
editarUsuario.php
conexion.php
funciones.php
funcionesFinalAhoraSi.php
```

¿Qué pasa cuando el  
sistema tiene  
300 funcionalidades?

Necesitamos ORGANIZAR

# Organizar = separar responsabilidades

---

¿Quién recibe la petición?

¿Quién decide qué hacer?

¿Quién trabaja con los datos?

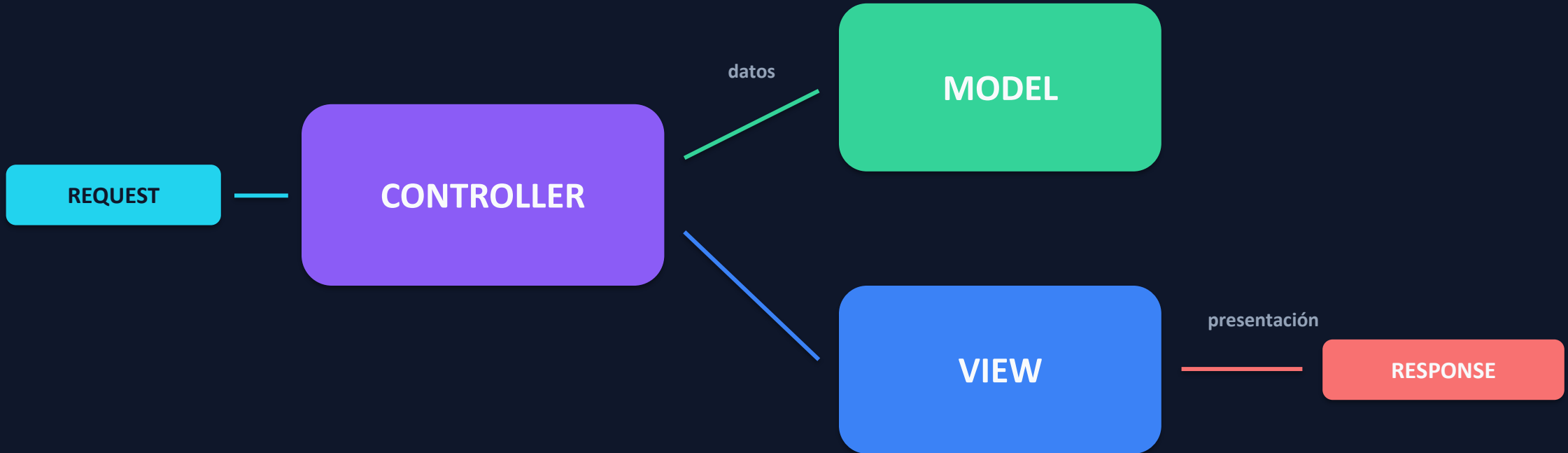
¿Quién construye lo que verá el  
usuario?

Una buena arquitectura hace explícitas estas decisiones.

# MVC

Una primera mirada. No hace falta memorizarlo hoy.

---



**¿Y vamos a programar  
todo esto desde cero?**

**NO.**

Hay problemas que la industria ya resolvió muchas veces.

Entonces aparece un framework.

# Laravel

**Una forma organizada de construir aplicaciones web con PHP.**

Rutas

MVC

ORM

Validación

Seguridad

APIs

Próximo episodio: no vamos a construir todo esto desde cero.

# Cierre · ¿Dónde ocurre cada cosa?

Cliente · Servidor · Base de datos · ¿más de uno?

---

Cambiar el color de un botón

Validar que un campo no esté vacío

Verificar una contraseña

Guardar una reserva

Listar alumnos almacenados

Mostrar un mensaje sin recargar toda la página

**No buscamos “adivinar”. Buscamos justificar.**